Alexandre Hurel *21231339*
Mayes Haddab *21206341*

LU3IN025 – Rapport Projet

3ème Projet

Affectation stable par l'algorithme de Gale-Shapley et PLNE

Fichier principal : `projet.py`

Sorbonne Université

L3 Informatique – Année 2025–2026

0. Contents

1 Reformulation du sujet **2**

2 Code et structures de données **2**
2.1 Organisation du code 2
2.2 Matrices principales (Q1) 2
2.3 Description schématique des algorithmes 2

3 Réponses aux questions **3**

4 Jeux d'essais **8**

5 Analyse des performances **8**
5.1 Temps de calcul 8
5.2 Nombre d'itérations 8
5.3 Conclusion 9

1. Reformulation du sujet

On cherche à affecter $n = 13$ **étudiants** dans $m = 10$ **parcours** du Master Informatique de Sorbonne Université (AI2D, BIM, CCA, IMA, MIND, QI, RES, SAR, SESI, STL), chaque parcours ayant une capacité d'accueil. Les deux côtés expriment des préférences ordonnées sur l'autre côté. L'objectif est de produire des **affectations stables** via l'algorithme de Gale-Shapley (côté étudiants, puis côté parcours), d'étudier leurs performances en temps et en nombre d'itérations, puis d'optimiser l'équité via la **Programmation Linéaire en Nombres Entiers (PLNE)** avec Gurobi.

2. Code et structures de données

2.1 Organisation du code

`projet.py` est structuré en **7 classes** :

Classe	Questions	Rôle
`PreferenceData`	Q1	Lecture de `PrefEtu.txt` / `PrefSpe.txt` ; construction de CE, CP et des matrices de rang
`GaleShapleyStudentSide`	Q2–Q3	GS côté étudiants avec max-heap
`GaleShapleySpeSide`	Q4–Q5	GS côté parcours
`StabilityChecker`	Q6	Détection des paires instables en $O(n \times m)$
`RandomGenerator`	Q7	Génération d'instances aléatoires
`PerformanceMeasurer`	Q8–Q10	Mesure du temps et des itérations, tracé des courbes
`PLNESolver`	Q11–Q14	Formulations PLNE résolues avec Gurobi

2.2 Matrices principales (Q1)

- **CE**$[i]$: liste des IDs de parcours par ordre de préférence de l'étudiant i (1er choix en tête).
- **CP**$[j]$: liste des IDs d'étudiants par ordre de préférence du parcours j.
- **rank_etu**$[i][j]$: rang du parcours j dans la liste de i (0 = meilleur), précalculé en $O(n \times m)$.
- **rank_spe**$[j][i]$: rang de l'étudiant i dans la liste de j, précalculé en $O(n \times m)$.

Ces matrices inverses permettent des comparaisons de préférences en $O(1)$ au lieu de $O(m)$ ou $O(n)$.

2.3 Description schématique des algorithmes

GS côté étudiants (student-optimal) :

```
Initialiser : file deque de tous les etudiants libres,
              next_proposal[i] = 0 pour tout i
Tant qu'il existe un etudiant libre i :
    j <- CE[i][next_proposal[i]++]
    Si |assignment[j]| < cap_j :
        affecter i -> j  (heappush dans worst_heap[j])
    Sinon :
        w <- pire de j (sommet du max-heap : rank_spe[j][w] maximal)
        Si rank_spe[j][i] < rank_spe[j][w] :   // j prefere i a w
            remplacer w par i dans j  (heapreplace en O(log cap_j))
            w redevient libre
        Sinon : i reste libre (proposera a CE[i][next_proposal[i]])
Retourner assignment
```

GS côté parcours (specialty-optimal) :

```
Initialiser : active = {tous les parcours},
              next_proposal[j] = 0, current_spe[i] = None
Tant qu'il existe un parcours actif j :
    active.discard(j)
    Tant que |assignment[j]| < cap_j et next_proposal[j] < n :
        i <- CP[j][next_proposal[j]++]
        Si current_spe[i] est None :
            affecter i -> j
        Sinon (i est dans j') :
            Si rank_etu[i][j] < rank_etu[i][j'] :  // i prefere j a j'
                deplacer i de j' vers j
                Si j' sous-rempli et a des candidats restants :
                    active.add(j')
            Sinon : j continue (passe au candidat suivant)
Retourner assignment
```

3. Réponses aux questions

Q2 – Structures de données pour GS côté étudiants

Opération	Structure	Complexité
Trouver un étudiant libre	`deque`	$O(1)$
Prochain parcours à proposer	`next_proposal[i]` (liste)	$O(1)$
Rang de i dans j	`rank_spe[j][i]` (tableau 2D précalculé)	$O(1)$
Pire étudiant de j	`worst_heap[j]` (max-heap via négation du rang)	$O(\log \text{cap}_j)$
Remplacer dans j	`heapreplace` + `set.add/remove`	$O(\log \text{cap}_j)$

La max-heap stocke $(-\texttt{rank_spe}[j][i], i)$: le minimum Python (sommet) correspond à l'étudiant que j classe en dernier parmi ses affectés actuels.

Occupation mémoire : $O(n \times m)$ pour les matrices de rang, $O(n)$ pour les structures auxiliaires.

Q3 – Algorithme GS côté étudiants et complexité

Chaque étudiant propose à au plus m parcours $\Rightarrow$ au plus $n \times m$ propositions, chacune coûtant $O(\log \text{cap}_{\max})$ pour les opérations de heap.

$$\textbf{Complexité totale :}\ O(n \times m \times \log(\text{cap}_{\max}))$$

Avec capacités équilibrées ($\text{cap} \approx n/m$) : $O(n \times m \times \log(n/m))$.

Sur l'instance réelle : **28 propositions** (pire cas théorique : $n \times m = 130$).

Q4 – Algorithme GS côté parcours et complexité

Chaque parcours propose à au plus n étudiants $\Rightarrow$ au plus $n \times m$ propositions, chacune en $O(1)$ grâce à `rank_etu`.

$$\textbf{Complexité totale :}\ O(n \times m)$$

Strictement meilleure que côté étudiant lorsque $\text{cap}_{\max}$ est grand.

Opération	Structure	Complexité
Parcours actif	`active` (set)	$O(1)$
Prochain étudiant à contacter	`next_proposal[j]` (liste)	$O(1)$
Comparer deux parcours pour i	`rank_etu[i][j]` (tableau 2D)	$O(1)$
Parcours actuel de i	`current_spe[i]` (liste)	$O(1)$
Modifier l'affectation	`assignment[j]` (set)	$O(1)$

Sur l'instance réelle : **57 propositions**.

Q5 – Application sur les fichiers de test

Les deux algorithmes produisent la **même affectation** (unicité de l'affectation stable sur cet exemple) :

Parcours	Cap.	Étudiants affectés
AI2D	2	Etu5, Etu12
BIM	1	Etu4
CCA	1	Etu9
IMA	1	Etu8
MIND	2	Etu10, Etu11
QI	1	Etu0
RES	1	Etu1
SAR	1	Etu7
SESI	1	Etu6
STL	2	Etu2, Etu3

Q6 – Vérification de la stabilité

Une paire (i, j) est instable si : $i \notin$ `assignment`$[j]$, i préfère j à son affectation courante, et j préfère i à l'un de ses étudiants (ou j a une place libre).

La méthode `StabilityChecker.find_unstable_pairs` procède en deux passes :

1. Construire `current_spe`$[i]$ (affectation inverse) en $O(n)$.
2. Pré-calculer `worst_rank_in`$[j]$ = rang maximum parmi les étudiants de j, en $O(n)$ au total.
3. Tester toutes les paires (i, j) : chaque test est $O(1)$.

Complexité : $O(n \times m)$.

Résultats :

- GS côté étudiants : **0 paire instable** ✓
- GS côté parcours : **0 paire instable** ✓

Q7 – Génération d'instances aléatoires

- `generate_CE(n, m)` : permutation aléatoire de $[0, m-1]$ pour chaque étudiant.
- `generate_CP(n, m)` : permutation aléatoire de $[0, n-1]$ pour chaque parcours.
- `generate_balanced_capacities(n, m)` : $\text{cap}_j = \lfloor n/m \rfloor$, les $n \bmod m$ premiers ont $\text{cap}_j + 1$; garantit $\sum \text{cap} = n$.

Q8 – Mesure des temps de calcul

Résultats moyens (10 tests par valeur de n, $m = 10$, capacités équilibrées) :

n	t_{etu} (ms)	t_{parcours} (ms)	iter. etu	iter. parcours
200	0.12	0.28	261	882
400	0.15	0.37	479	1 820
600	0.22	0.59	710	2 900
800	0.29	0.82	931	4 038
1 000	0.37	1.06	1 126	5 177
1 200	0.41	1.27	1 349	6 240
1 400	0.51	1.53	1 571	7 423
1 600	0.55	1.80	1 776	8 693
1 800	0.58	1.96	1 966	9 597
2 000	0.67	2.27	2 180	10 905

Courbes sauvegardées : `performance_times.png` (temps) et `performance_iters.png` (itérations).

Q9 – Complexité observée

Les temps croissent **linéairement** en n pour les deux algorithmes ($m = 10$ fixé), conformément à :

- GS côté étudiants : $O(n \times m \times \log(n/m)) \approx O(n \log n) \approx O(n)$ pour m constant.
- GS côté parcours : $O(n \times m) = O(n)$ pour m constant.

Le ratio des temps $t_{\text{spe}}/t_{\text{etu}} \approx 3{,}4\times$ à $n = 2\,000$ s'explique directement par le ratio d'itérations $\approx 5{,}0\times$: chaque itération côté parcours est certes en $O(1)$ (plus rapide que $O(\log \text{cap})$), mais le nombre total de propositions est bien supérieur.

Q10 – Nombre d'itérations

Les itérations croissent également **linéairement** en n :

- Côté étudiants : $\approx 1{,}1n$ propositions en moyenne (la plupart des étudiants trouvent rapidement une place).
- Côté parcours : $\approx 5{,}4n$ propositions en moyenne (phénomène analogue au problème du coupon collector : chaque parcours doit parcourir plus d'étudiants pour remplir son quota face aux refus).

À $n = 2\,000$: côté etu $\approx 11\%$ du pire cas ($n \times m = 20\,000$), côté parcours $\approx 55\%$ – tous deux restent bien en-deçà du pire cas théorique.

Q11 – PLNE maximisant l'utilité minimale des étudiants

Le **score de Borda** de l'étudiant i affecté au parcours j est défini par :

$$b_{ij} = m - 1 - \texttt{rank_etu}[i][j] \ \in \ \{0, \ldots, m-1\}.$$

Variables : $x_{ij} \in \{0, 1\}$ indique si l'étudiant i est affecté au parcours j ; $U_{\min} \geq 0$ est l'utilité minimale à maximiser.

Maximiser $U_{\min}$

Sous les contraintes :

$$U_{\min} \leq \sum_j b_{ij} \cdot x_{ij}, \quad \forall i \quad \text{(chaque étudiant a une utilité} \geq U_{\min})$$

$$\sum_j x_{ij} = 1, \quad \forall i \quad \text{(chaque étudiant est affecté à exactement un parcours)}$$

$$\sum_i x_{ij} \leq \text{cap}_j, \quad \forall j \quad \text{(respect de la capacité de chaque parcours)}$$

$$x_{ij} \in \{0, 1\}, \quad \forall i, j$$

Résultat Gurobi : $U_{\min}$ optimal = **5** (vs 4 pour GS), utilité moyenne = 8,23 ; 3 paires instables.

Q12 – PLNE maximisant la somme des utilités

On définit le score de Borda du parcours j pour l'étudiant i par :

$$c_{ji} = n - 1 - \texttt{rank_spe}[j][i].$$

Maximiser $\sum_{i,j} (b_{ij} + c_{ji}) \cdot x_{ij}$

Sous les contraintes :

$$\sum_j x_{ij} = 1, \quad \forall i \quad \text{(chaque étudiant est affecté à exactement un parcours)}$$

$$\sum_i x_{ij} \leq \text{cap}_j, \quad \forall j \quad \text{(respect de la capacité de chaque parcours)}$$

$$x_{ij} \in \{0, 1\}, \quad \forall i, j$$

Résultat Gurobi : utilité totale = **213**, utilité moyenne étudiants = 7,77, minimum = 4 ; 7 paires instables.

Q13 – PLNE avec contrainte des k premiers choix

Un étudiant i est dans ses k premiers choix si et seulement si son score de Borda vérifie :

$$b_{ij} \geq m - k \iff \texttt{rank_etu}[i][j] \leq k - 1.$$

Maximiser $\sum_{i,j}(b_{ij} + c_{ji}) \cdot x_{ij}$

Sous les contraintes :

$$\sum_j x_{ij} = 1, \qquad \forall i \quad \text{(chaque étudiant est affecté à exactement un parcours)}$$

$$\sum_i x_{ij} \leq \text{cap}_j, \qquad \forall j \quad \text{(respect de la capacité de chaque parcours)}$$

$$\sum_j b_{ij} \cdot x_{ij} \geq m - k, \qquad \forall i \quad \text{(chaque étudiant est affecté dans ses } k \text{ premiers choix)}$$

$$x_{ij} \in \{0, 1\}, \qquad \forall i, j$$

Q14 – Plus petit k pour un mariage parfait

On résout Q13 pour $k = 1, 2, 3, \ldots$ jusqu'à la première faisabilité :

k	**Faisable ?**	**Util. min étu.**
1	Non	–
2	Non	–
3	Non	–
4	Non	–
5	**Oui**	**5**

Plus petit $k = 5$. Affectation : AI2D = {Etu7, Etu12}, BIM = {Etu4}, CCA = {Etu9}, IMA = {Etu10}, MIND = {Etu2, Etu11}, QI = {Etu1}, RES = {Etu8}, SAR = {Etu6}, SESI = {Etu0}, STL = {Etu3, Etu5}.

Utilité moyenne = 7,85, minimum = 5 ; 5 paires instables.

Q15 – Comparaison des affectations

Affectation	**Util. moy. etu**	**Util. min. etu**	**Paires instables**
GS côté étudiants	7,85	4	**0** (stable)
GS côté parcours	7,85	4	**0** (stable)
Q11 – maximin	**8,23**	**5**	3
Q12 – max somme	7,77	4	7
Q14 – $k = 5$, max somme	7,85	**5**	5

Analyse :

- Les deux GS donnent la même affectation (unique affectation stable ici) et sont les **seules stables**. Elles constituent donc l'optimum parmi les affectations stables.
- **Q11** améliore significativement l'équité (utilité min 4 → 5, +25%) au prix de 3 paires instables.
- **Q12** maximise le bien-être global (étudiants + parcours = 213) mais n'améliore pas l'utilité minimale par rapport à GS, avec 7 paires instables – le moins bon compromis.

- **Q14** ($k = 5$) est le meilleur compromis : même utilité moyenne que GS, même utilité min que Q11, avec une contrainte d'équité explicite (chaque étudiant dans ses 5 premiers choix).

4. Jeux d'essais

Jeu d'essai	Description
`PrefEtu.txt` + `PrefSpe.txt`	Instance réelle : 13 étudiants, 10 parcours, capacités $[2, 1, 1, 1, 2, 1, 1, 1, 1, 2]$
Instances aléatoires (Q8–Q10)	$n \in \{200, 400, \ldots, 2\,000\}$, $m = 10$, capacités équilibrées, 10 tirages par n

Validation : `StabilityChecker` retourne 0 paire instable sur toutes les instances GS testées ; $\sum \text{cap} = n$ vérifié systématiquement ; chaque étudiant est affecté à exactement un parcours et les capacités sont respectées.

5. Analyse des performances

5.1 Temps de calcul

La figure 1 présente le temps de calcul moyen des deux algorithmes en fonction de n, sur 10 tirages par valeur de n.

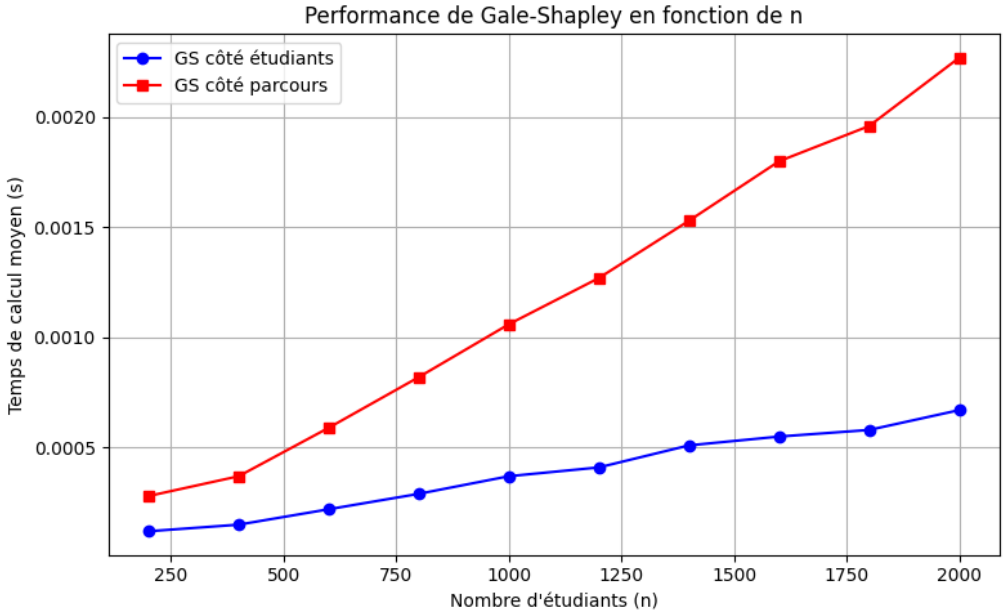

Figure 1: Temps de calcul moyen de Gale-Shapley (côté étudiants vs côté parcours) en fonction du nombre d'étudiants n, $m = 10$, capacités équilibrées, moyenne sur 10 tirages.

On observe une croissance **linéaire** en n pour les deux variantes, conformément à la théorie. À $n = 2\,000$, GS côté parcours est $\approx 3{,}4\times$ plus lent (2,27 ms vs 0,67 ms) malgré une complexité asymptotique inférieure – l'overhead pratique vient du nombre de propositions $\approx 5\times$ supérieur.

5.2 Nombre d'itérations

La figure 2 présente le nombre moyen d'itérations (propositions) effectuées par chacun des deux algorithmes.

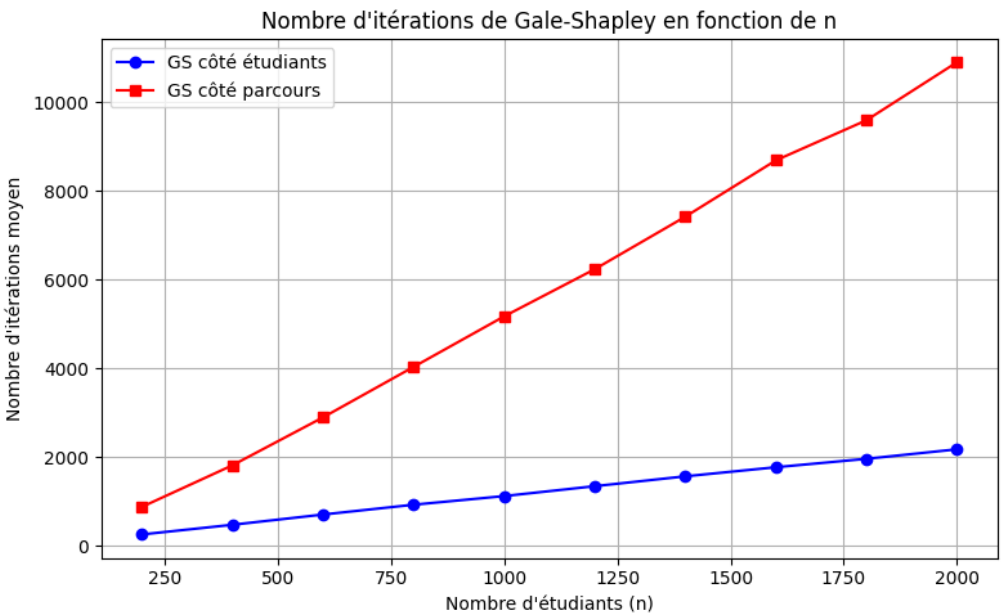

Figure 2: Nombre d'itérations moyen de Gale-Shapley (côté étudiants vs côté parcours) en fonction du nombre d'étudiants n, $m = 10$, moyenne sur 10 tirages.

On observe également une croissance **linéaire** en n, bien en-deçà du pire cas $n \times m$. La version étudiants est nettement plus économique ($\approx 1{,}1n$ vs $5{,}4n$ propositions) car la max-heap évite les aller-retours superflus entre étudiants libres et parcours pleins.

5.3 Conclusion

GS côté étudiants est plus efficace en pratique ; GS côté parcours est conceptuellement plus simple ($O(1)$ par proposition) mais compense en volume. Pour des instances de grande taille avec peu de parcours ($m \ll n$), la version étudiants reste préférable.